NumericalAlgebra 1

王胤雅 Yinya Wang

25114020018

yinyawang25@m.fudan.edu.cn

2026 年 4 月 2 日

PROBLEM I $H_{n+1,n}$ is the Hessenberg matrix of Arnoldi process, which is executed for A and b at step n. If $h_{N+1,N} = 0$, then

1. $\mathrm{K}_N(A, b)$ is $A-$ invariant subspace.
2. $\mathrm{K}_n(A, b) = \mathrm{K}_{n+1}(A, b), n \geq N$.
3. $\sigma(H_N) \subset \sigma(A)$, where $\sigma(M)$ is the eigenvalue set of matrix M.
4. If A is nonsigular, the solution of the equation $Ax = b$ lies in $\mathrm{K}_N(A, b)$.

SOLUTION. Assume $Q_n = (q_1, \cdots, q_n)$ at step n in Arnoldi process.

1. Since $K_{N+1}(A, b) = span\{b, \cdots, A^N b\} = span\{b, AK_N(A, b)\}$, we only need to prove $AK_N(A, b) \subset K_N(A, b)$. Since $AQ_n = Q_{n+1}H_{n+1,n}$ exists for all step n and $H_{N+1,N} = 0$, we can get $AQ_N = Q_N H_N + h_{N+1,N} q_{N+1} e_N^T$, where e_N is unit vector with only one none zero element at position N. Thus,
$$AQ_N = Q_N H_N. \tag{1}$$
Besides, $\{q_1, \cdots, q_N\}$ is the base of space $K_N(A, q)$, so $span\{Aq_1, \cdots, A_N\} = AK_N(A, b) = span(Q_N H_N) \subset K_N(A, q)$.

2. First, we can prove $K_n(A, b) \subset K_N(A, b), n \geq N$. We will use MI to prove. $n = N$, this property holds, obviously. Assume $K_t(A, b) \subset K_N(A, b)$ for some $t \geq N$, then $K_{t+1}(A, b) = span\{b, AK_t(A, b)\}$. Besides, $K_N(A, b)$ is A- invariant subspace, then $AK_t(A, b) \subset AK_N(A, b) \subset K_N(A, b)$. Thus, $K_{t+1}(A, b) \subset K_N(A, b)$.

   Since $K_n(A, b) = span\{b, \cdots, A^{n-1}b\} \supset span\{b, \cdots, A^{N-1}b\} = K_N(A, b)$, we can get $K_n(A, b) = K_N(A, b), \forall n \geq N$.

3. Assume λ is the eigenvalue of H_N with eigenvector u, then $H_N u = \lambda u$. By (1), $AQ_N u = Q_N H_N u = \lambda Q_N u$, therefore, λ is an eigenvalue of A with eigenvector $Q_N u$.

4. We only need to prove $A|_{K_N(A,b)} : K_N(A,b) \to K_N(A,b)$ is bijective. Since $K_N(A,b)$ is A-invariant, we can get $A|_{K_N(A,b)}$ well-defined. $\forall y \in K_N(A,b), Ay = 0$, then $y = 0$ due to the reversibility of A. Thus, $Ker(A|_{K_N(A,b)}) = 0$, then $A|_{K_N(A,b)}$ is injective. Since $\{q_1, \cdots, q_N\}$ are base of $K_N(A,b)$, we can get $\sum_{i=1}^N c_i q_i = 0 \iff c_i = 0, 1 \le i \le N$. Then $\sum_{i=1}^N d_i A q_i = A(\sum_{i=1}^N d_i q_i) = 0 \iff \sum_{i=1}^N d_i q_i = 0 \iff d_i = 0, 1 \le i \le N$. Then, $\{Aq_1, \cdots, Aq_N\}$ are base of $K_N(A,b)$. We can get that $\forall y \in K_N(A,b)$, $y = \sum_{i=1}^N a_i A q_i = A \sum_{i=1}^N a_i q_i = Ax$, where $x = \sum_{i=1}^N a_i q_i$. So $A|_{K_N(A,b)}$ is surjective. Just let $y = b$, the equation has solution in $K_N(A,b)$.

□

PROBLEM II Let $A \in \mathbb{C}^{m\times m}$, $b \in \mathbb{C}^m$, and $p \in \mathcal{P}_n$. Suppose that we want to compute $p(A)b$. A natural place to start is with Horner's rule, which can be written as

$$p(z) = c_0 + z\big(c_1 + z(c_2 + \cdots + z(c_{n-1} + zc_n)\cdots)\big). \tag{2}$$

(a) Write a `for` loop based on (2) (on paper, not on a computer) that computes $p(A)$ and then applies the result to b. Determine the number of flops required by this algorithm, to leading order.

(b) Write another `for` loop for computing $p(A)b$, a far better one, in which b is introduced into the process at the beginning. Again determine the number of flops required to leading order.

(c) In introductory numerical analysis texts, Horner's rule is recommended for evaluation of polynomials because it is faster than the obvious method of computing powers z^k, multiplying by coefficients c_k, and adding. Show that for computing $p(A)$ or $p(A)b$, by contrast, Horner's rule is not significantly faster than the obvious method.

SOLUTION. (a) A straightforward implementation of Horner's rule for first computing $p(A)$ and then applying it to b is:

Algorithm 1 Horner method for computing $p(A)$ and then $p(A)b$

```
P ← c_n I_m
for k = n−1, n−2, ..., 0 do
    P ← AP + c_k I_m
end for
y ← Pb
return y
```

At each step of the loop, the dominant operation is the matrix–matrix product AP, which costs $2m^3 + O(m^2)$ flops. The addition $c_k I_m$ changes only the diagonal entries, so it costs only m flops. Thus each iteration costs

$$2m^3 + O(m^2)$$

flops, and over n iterations the total cost is

$$2nm^3 + O(nm^2).$$

Finally, multiplying P by b costs $2m^2 + O(m)$ flops. Therefore the total number of flops is

$$2nm^3 + O(nm^2),$$

so to leading order the algorithm requires

$$\boxed{2nm^3 \text{ flops.}}$$

(b) A much better algorithm is to introduce b at the beginning and evaluate $p(A)b$ directly:

Algorithm 2 Horner method for directly computing $p(A)b$

```
y ← c_n b
for k = n − 1, n − 2, ..., 0 do
    y ← Ay + c_k b
end for
return y
```

Now the dominant operation in each iteration is the matrix–vector product Ay, which costs $2m^2 + O(m)$ flops. The vector update $y + c_k b$ costs only $O(m)$ flops. Hence each iteration costs

$$2m^2 + O(m)$$

flops, and after n iterations the total is

$$2nm^2 + O(nm).$$

Thus, to leading order, this algorithm requires

$$\boxed{2nm^2 \text{ flops.}}$$

This is far better than part (a), since it reduces the cost from order m^3 per step to order m^2 per step.

(c) The "obvious" method for computing $p(A)$ is to form the powers A^k, multiply each by c_k, and add:

$$p(A) = c_0 I + c_1 A + c_2 A^2 + \cdots + c_n A^n.$$

This requires computing the powers $A^2, A^3, \ldots, A^n$, which involves $n-1$ matrix–matrix multiplications. Since each such multiplication costs $2m^3 + O(m^2)$ flops, the total cost is again

$$2nm^3 + O(nm^2),$$

i.e.,

$$\boxed{O(nm^3)}$$

to leading order. Therefore Horner's rule is not significantly faster than the obvious method for computing $p(A)$: both have the same leading-order cost.

Likewise, the obvious method for computing $p(A)b$ is to form the vectors

$$b,\ Ab,\ A^2b,\ \ldots,\ A^nb$$

recursively, and then combine them:

$$p(A)b = c_0 b + c_1 Ab + c_2 A^2 b + \cdots + c_n A^n b.$$

Computing $A^k b$ successively requires n matrix–vector products, each costing $2m^2 + O(m)$ flops, so the total cost is

$$2nm^2 + O(nm),$$

i.e.,

$$\boxed{O(nm^2)}$$

to leading order. This is the same leading-order cost as the Horner method in part (b).

Hence, unlike the scalar case, Horner's rule is not significantly faster than the obvious method when computing $p(A)$ or $p(A)b$. The reason is that in the matrix setting the dominant cost comes from matrix–matrix or matrix–vector multiplications, and both approaches require essentially the same number of such expensive operations.

□

PROBLEM III $H \in \mathbb{C}^{mxm}$ is a Hessenberg matrix, f is a polynomial with degree $n < m$, $H_n \in \mathbb{C}^{nxn}$ is the first matrix block of H. Then $f(H)$ and $f(H_n)$ has same n entries in the first column.

SOLUTION. Since $A = QHQ^*$, we have

$$p_n(A)b = Qp_n(H)Q^*b.$$

Because $q_1 = b/\|b\|$, it follows that

$$Q^*b = \|b\|e_1.$$

Hence

$$Q_n^* p_n(A)b = \|b\|\, Q_n^* Q\, p_n(H)e_1 = \|b\|\, [I_n\ 0]\, p_n(H)e_1.$$

Therefore $Q_n^* p_n(A)b = 0$ if and only if the first n entries of the first column of $p_n(H)$ are zero.

Now write

$$p_n(z) = \sum_{k=0}^{n} a_k z^k.$$

Since H is upper Hessenberg, for each $k \le n$, the first n entries of $H^k e_1$ depend only on the leading principal block H_n, and equal $H_n^k e_1$. Therefore

$$[I_n\ 0]\, p_n(H)e_1 = \sum_{k=0}^{n} a_k [I_n\ 0]\, H^k e_1 = \sum_{k=0}^{n} a_k H_n^k e_1 = p_n(H_n)e_1.$$

Thus the first n entries of the first column of $p_n(H)$ are exactly the first n entries of the first column of $p_n(H_n)$. □

PROBLEM IV Assume H_n at step n in Arnoldi process, the Ritz values of H_n are eigenvalues of H_n.

1. If A is changed to $A + \sigma I$,$\sigma \in \mathbb{C}$,b is unchanged, then the Ritz values $\{\theta_j\}$ are changed to $\{\theta_j + \sigma\}$.

2. If A is changed to σA,$\sigma \in \mathbb{C} \setminus \{0\}$,$b$ is unchanged, then the Ritz values $\{\theta_j\}$ are changed to $\{\sigma\theta_j\}$.

3. $U^*U = I$, if A is changed to UAU^*,b is changed to Ub, then the Ritz values $\{\theta_j\}$ don't change.

SOLUTION. 1. Since $(A+\sigma I)^k = \sum_{t=0}^{k} a_t A^t \subset K_n(A,b), 1 \le k \le n$, we can get that $K_n(A+\sigma I, b) = span\{b, (A+\sigma I)b, \cdots, (A+\sigma I)^{n-1}b\} \subset span\{b, Ab, \cdots, A^{n-1}b\} = K_n(A,b)$. Besides, $(A+\sigma I - \sigma I) = A$, then $K_n((A+\sigma I)-\sigma I, b) \subset K_n(A+\sigma I, b)$ holds for the same reason. Thus, $K_n(A+\sigma I, b) = K_n(A,b)$. Assume Q_n, H_n be the responding matrix in Arnoldi process executed by A and b at step n. So Q_n is the orthonormal basis of $K_n(A+\sigma I, b)$. Then $AQ_n = Q_{n+1}H_{n+1} = Q_nH_n + h_{n+1,n}q_{n+1}e_n^T$, then $Q_n^*AQ_n = Q_n^*Q_nH_n + h_{n+1,n}Q_n^*q_{n+1}e_n^T = H_n$. Then $\tilde{H}_n = Q_n^*(A+\sigma I)Q_n = Q_n^*AQ_n + \sigma I = H_n + \sigma I$, where $\tilde{H}_n$ is the representation matrix of $K_n(A+\sigma I, b)$ in base Q_n.

Assume f be the characteristic polynomial of H_n, $\tilde{f}$ be the characteristic polynoimal of $\tilde{H}_n$. So $f(\lambda) = |H_n - \lambda I| = |\tilde{H}_n - \sigma I - \lambda I| = \tilde{f}(\lambda+\sigma)$. Thus, θ be the Ritz of H_n, then $f(\theta) = 0 = \tilde{f}(\theta+\sigma)$. Then $\theta+\sigma$ is the Ritz of $\tilde{H}_n$.

2. Since $(\sigma A)^k = \sigma^k A^k \in K_n(A,b), 1 \le k \le n$, we can get that $K_n(\sigma A, b) = span\{b, \sigma Ab, \cdots, (\sigma A)^{n-1}b\} \subset K_n(A,b)$. Besides, $\sigma^{-1}(\sigma A) = A$, then $K_n(\sigma^{-1}(\sigma A), b) \subset K_n(\sigma A, b)$ holds for the same reason. Thus, $K_n(\sigma A, b) = K_n(A,b)$. In the same reason as above, we can get $\tilde{H}_n = Q_n^*(\sigma A)Q_n = \sigma H_n$, where $\tilde{H}_n$ is the representation matrix of $K_n(\sigma A, b)$ in orthonormal basis Q_n.

Assume f be the characteristic polynomial of H_n, $\tilde{f}$ be the characteristic polynoial of $\tilde{H}_n$. So $f(\lambda) = |H_n - \lambda I| = |(\sigma)^{-1}(\tilde{H}_n - \sigma\lambda I)| = \sigma^{-n}|\tilde{H}_n - \sigma\lambda I| = \sigma^{-n}\tilde{f}(\sigma\lambda)$. Thus, θ be the Ritz of H_n, then $f(\theta) = 0 = \sigma^{-n}\tilde{f}(\sigma\theta)$. Then $\tilde{\sigma\theta} = 0$. Then $\sigma\theta$ is the Ritz of $\tilde{H}_n$.

3. Since $(UAU^*)^k(Ub) = UA^kU^*Ub = UA^kb$, then $K_n(UAU^*, Ub) = UK_n(A,b)$. Besides, U is reversible, so UQ_n is orthonormal matrix. Thus, UQ_n is the orthonormal basis of $K_n(UAU^*, Ub)$. Then $\tilde{H}_n = (UQ_n)^*(UAU^*)(UQ_n) = Q_n^*AQ_n = H_n$. Then, Ritz values don't change.

□

PROBLEM V $P_n := \{1 + \sum_{i=1}^{n} c_i z^i : c_i \in \mathbb{C}, 1 \le i \le n\}$, $q = \text{argmin}_{p \in P_n} \|p(A)b\|$, $r = q(A)b$.

1. If A changed to σA,$\sigma \in \mathbb{C} \setminus \{0\}$,$b$ is changed to σb, then r is changed to r.

2. $U^*U = I$, if A is changed to UAU^*,b is changed to Ub, then r changed to Ur.

SOLUTION. 1. Assume $Q_n := \{\sigma p(\sigma z) : p \in P_n\}$, $f : P_n \to Q_n, p(z) \mapsto \sigma p(\sigma z)$. We prove that f is bijective. Obviously, f is well-defined and surjective. Only need to prove f is injective. If $\sigma p(\sigma z) = \sigma q(\sigma z)$, then $p(\sigma z) = q(\sigma z)$. Thus, $p(z) = q(z)$, which means f is injective. Since $q = \text{argmin}_{p \in P_n} \|p(A)b\|$ and $q(A)b = \sigma^{-1}(q(\sigma^{-1}(\sigma A))(\sigma b)) = \tilde{q}(\sigma A)(\sigma b)$, where $\tilde{q}(z) = \sigma^{-1}q(\sigma^{-1}z)$. Thus, $\tilde{q} = \text{argmin}_{p \in Q_n} \|p(\sigma A)(\sigma b)\|$. So $\tilde{r} = \tilde{q}(\sigma A)(\sigma b) = \sigma^{-1}q(\sigma^{-1}(\sigma A))(\sigma b) = q(A)b = r$.

2. First, $\forall x \in \mathbb{C}^m$, then $\|Ux\|^2 = (Ux)^*Ux = x^*U^*Ux = x^*x = \|x\|^2$. Besides, $\forall p \in P_n$, $p(UAU^*)(Ub) = Up(A)U^*Ub = Up(A)b$, then $\|p(UAU^*)(Ub)\| = \|Up(A)b\| = \|p(A)b\|$. Therefore, $q = \text{argmin}\|p(A)b\| = \text{argmin}\|Up(A)b\|$. Thus, $q = \tilde{q}$. Therefore, $\tilde{r} = \tilde{q}(UAU^*)(Ub) = U\tilde{q}(A)b = Uq(A)b = Ur$.

□

PROBLEM VI

(a) Let $S \subset \mathbb{C}$ be a set whose convex hull contains 0 as an interior point. That is, S is contained in no half-plane disjoint from the origin. Show that there is no $p \in \mathcal{P}_1$ (i.e., no polynomial p of degree 1 with $p(0) = 1$ such that

$$\sup\{|p(z)| : z \in S\} < 1.$$

(b) Let A be a matrix, not necessarily normal, whose spectrum $\sigma(A)$ has the property described in part (a). Show that there is no $p \in \mathcal{P}_1$ such that

$$\|p(A)\| < 1.$$

(c) Describe what kind of polynomial $p_1 \in \mathcal{P}_1$ GMRES has probably found to achieve

$$\|r_1\| < \|r_0\|,$$

and explain why this does not contradict the result of part (b).

SOLUTION. 1. First, we will prove $\forall p \in P_1$, $\mathcal{A} = \{z : |p(z)| < 1, z \in \mathbb{C}\}$ is convex set. Assume $p = 1 + az, a \in \mathbb{C}$. $\forall z \in \mathcal{A}$, then $|p(z)| = \overline{p(z)}p(z) = (1+az)\overline{(1+az)} = 1 + \overline{az} + az + |az|^2 < 1$, that is $2Re(az) + |az|^2 < 0$.

$\forall z_1, z_2 \in \mathcal{A}$, $\lambda \in [0, 1]$, then

$$\begin{aligned}
|1 + a(\lambda z_1 + (1-\lambda)z_2)|^2 &= 1 + 2Re(\lambda a z_1 + (1-\lambda)az_2) + |\lambda a z_1 + (1-\lambda)az_2|^2 \\
&= 1 + 2Re(\lambda a z_1) + 2Re((1-\lambda)az_2) + |\lambda a z_1 + (1-\lambda)az_2|^2 \\
&= 1 - \lambda|az_1|^2 - (1-\lambda)|az_2|^2 + |\lambda a z_1 + (1-\lambda)az_2|^2 \\
&< 1 - \lambda a z_1 \overline{az_1} - (1-\lambda)az_2\overline{az_2} + (\lambda a z_1 + (1-\lambda)az_2)\overline{(\lambda a z_1 + (1-\lambda)az_2)} \\
&= 1 + \lambda(1-\lambda)(az_1 - az_2)\overline{az_2 - az_1} \\
&= 1 - \lambda(1-\lambda)|az_1 - az_2|^2 \leq 1
\end{aligned} \tag{3}$$

Therefore, $|1 + a(\lambda z_1 + (1-\lambda)z_2)| < 1$, so $\lambda z_1 + (1-\lambda)z_2 \in \mathcal{A}$. Now suppose, for contradiction, that

$$\sup_{z \in S} |p(z)| < 1.$$

Then $S \subset \Omega$. Since Ω is convex, it follows that

$$\text{conv}(S) \subset \Omega.$$

But 0 is an interior point of $\mathrm{conv}(S)$, so in particular $0 \in \mathrm{conv}(S) \subset \Omega$. This implies

$$|p(0)| < 1.$$

However, $p(0) = 1$, so $|p(0)| = 1$, a contradiction.

Therefore there is no $p \in \mathcal{P}_1$ such that

$$\sup\{|p(z)| : z \in S\} < 1.$$

2. Suppose there exists $p \in \mathcal{P}_1$ such that

$$\|p(A)\| < 1.$$

Then every eigenvalue $\lambda \in \sigma(A)$ of A gives rise to the eigenvalue $p(\lambda) \in \sigma(p(A))$. Hence

$$\sigma(p(A)) = p(\sigma(A)).$$

Since the spectral radius is bounded by the operator norm, we have

$$|p(\lambda)| \leq \rho(p(A)) \leq \|p(A)\| < 1 \qquad \text{for all } \lambda \in \sigma(A).$$

Therefore

$$\sup_{\lambda \in \sigma(A)} |p(\lambda)| < 1.$$

But $\sigma(A)$ has the property described in part (a), namely that 0 is an interior point of $\mathrm{conv}(\sigma(A))$. Applying part (a) with $S = \sigma(A)$, we conclude that no such $p \in \mathcal{P}_1$ can exist. This contradiction shows that there is no $p \in \mathcal{P}_1$ such that

$$\|p(A)\| < 1.$$

3. After one step, GMRES produces a residual of the form

$$r_1 = p_1(A) r_0,$$

where $p_1 \in \mathcal{P}_1$. Since p_1 is chosen by GMRES to minimize the residual norm, it is probably a polynomial for which

$$\|p_1(A) r_0\| < \|r_0\|.$$

In other words, GMRES has likely found a degree-one polynomial that reduces the norm of r_0 in this particular direction, even though it does not reduce the operator norm on all vectors.

This does not contradict part (b), because part (b) states that there is no $p \in \mathcal{P}_1$ such that

$$\|p(A)\| < 1,$$

i.e., no degree-one polynomial that contracts *every* vector uniformly. GMRES, however, only requires that

$$\|p_1(A) r_0\| < \|r_0\|$$

for the single vector r_0. It is entirely possible that

$$\|p_1(A)r_0\| < \|r_0\| \qquad \text{but} \qquad \|p_1(A)\| \geq 1.$$

Thus GMRES may succeed in decreasing the residual after one step even though no degree-one polynomial can make $p(A)$ a strict contraction in operator norm.

□

PROBLEM VII Our statement of the GMRES algorithm begins with the initial guess

$$x_0 = 0, \quad r_0 = b.$$

Show that if one wishes to start with an arbitrary initial guess x_0, this can be accomplished by an easy modification of the right-hand side b.

SOLUTION. Consider the linear system

$$Ax = b.$$

The GMRES algorithm is typically stated with the initial guess

$$x_0 = 0, \quad r_0 = b.$$

Now suppose we wish to start from an arbitrary initial guess $x_0 \neq 0$. Define the error variable

$$y = x - x_0.$$

Then substituting $x = x_0 + y$ into the original system gives

$$A(x_0 + y) = b.$$

Expanding this, we obtain

$$Ax_0 + Ay = b,$$

which can be rewritten as

$$Ay = b - Ax_0.$$

Define the modified right-hand side

$$\tilde{b} = b - Ax_0.$$

Then we obtain a new linear system

$$Ay = \tilde{b},$$

with initial guess

$$y_0 = 0.$$

Therefore, applying GMRES to the system $Ay = \tilde{b}$ with initial guess $y_0 = 0$ is equivalent to applying GMRES to the original system $Ax = b$ with initial guess x_0.

Finally, the solution to the original problem is recovered by

$$x = x_0 + y.$$

This shows that starting from an arbitrary initial guess x_0 can be achieved by modifying the right-hand side b to $b - Ax_0$. □

PROBLEM VIII For larger values of n, the cost of GMRES in operations and storage may be prohibitive. In such circumstances, a method called k-step restarted GMRES, or GMRES(k), is often employed. In this method, after k steps, the GMRES iteration is restarted with the current vector x_k as the new initial guess.

(a) Compare the asymptotic operation counts and storage requirements of GMRES and GMRES(k), for fixed k and increasing n.

(b) Describe an example in which GMRES(k) can be expected to converge in nearly as few iterations as GMRES (hence much faster in operation count).

(c) Describe another example in which GMRES(k) can be expected to fail to converge, whereas GMRES succeeds.

SOLUTION. 1. For standard GMRES, the Arnoldi process requires orthogonalization against all previous Krylov vectors. At iteration k, this leads to a cost of $O(nk)$ per iteration, and hence a total cost of $O(nk^2)$ after k iterations. In the worst case, $k = n$, yielding a total cost of $O(n^3)$.

The storage requirement is $O(nk)$, which becomes $O(n^2)$ in the worst case.

For GMRES(k), the iteration is restarted every k steps. Each cycle costs $O(nk^2)$, and if m total iterations are required, the total cost is $O(nkm)$. Since k is fixed, this scales linearly with n.

The storage requirement is reduced to $O(nk)$.

Thus, GMRES(k) has significantly lower storage requirements and computational cost for large n when k is fixed.

2. GMRES(k) performs well when the Krylov subspace of dimension k captures most of the important spectral information of A. This typically occurs when:

- The matrix A has a small number of distinct eigenvalues;
- The eigenvalues are well clustered;
- The system is well-conditioned.

In such cases, the minimal polynomial of A is of low degree, and GMRES converges rapidly within a small subspace. Therefore, restarting does not significantly affect convergence, and GMRES(k) achieves nearly the same convergence rate as full GMRES.

3. GMRES(k) may fail to converge when the problem requires a large Krylov subspace to capture the essential features of the matrix A. This often occurs for non-normal matrices.

A classical example is a Jordan block matrix:

$$A = \begin{bmatrix} \lambda & 1 & 0 & \cdots \\ 0 & \lambda & 1 & \cdots \\ \vdots & & \ddots & \ddots \end{bmatrix}.$$

In such cases, the convergence of GMRES depends on building up information over many iterations. Restarting every k steps discards previously accumulated information, preventing convergence.

Thus, GMRES(k) may stagnate, while full GMRES (without restarting) converges.

□

PROBLEM IX

(a) Show that the eigenvalues of a symmetric matrix $A \in \mathbb{R}^{m\times m}$ are stationary values of the Rayleigh quotient

$$r(x) = \frac{x^T Ax}{x^T x}.$$

(b) Show that the Ritz values at step n of the Lanczos iteration are stationary values of $r(x)$ when x is restricted to the Krylov subspace $\mathcal{K}_n$.

SOLUTION. Let

$$r(x) = \frac{x^T Ax}{x^T x}, \qquad x \neq 0,$$

where $A \in \mathbb{R}^{m\times m}$ is symmetric.

1. We compute the gradient of $r(x)$. Define

$$f(x) = x^T Ax, \qquad g(x) = x^T x.$$

Then

$$r(x) = \frac{f(x)}{g(x)}.$$

Since A is symmetric,

$$\nabla f(x) = 2Ax, \qquad \nabla g(x) = 2x.$$

By the quotient rule,

$$\nabla r(x) = \frac{g(x)\nabla f(x) - f(x)\nabla g(x)}{g(x)^2} = \frac{(x^T x)(2Ax) - (x^T Ax)(2x)}{(x^T x)^2}.$$

Hence

$$\nabla r(x) = \frac{2}{(x^T x)^2}\Big((x^T x)Ax - (x^T Ax)x\Big).$$

A stationary point satisfies $\nabla r(x) = 0$, so

$$(x^T x)Ax - (x^T Ax)x = 0.$$

Dividing by $x^T x \neq 0$, we obtain

$$Ax = \frac{x^T Ax}{x^T x}x = r(x)x.$$

Therefore, any stationary vector x of the Rayleigh quotient is an eigenvector of A, and the corresponding stationary value $r(x)$ is the associated eigenvalue.

Conversely, if x is an eigenvector of A with eigenvalue λ, then

$$Ax = \lambda x,$$

and thus

$$r(x) = \frac{x^T Ax}{x^T x} = \frac{x^T(\lambda x)}{x^T x} = \lambda.$$

Substituting $Ax = \lambda x$ into the formula for $\nabla r(x)$ gives

$$\nabla r(x) = 0.$$

Hence the stationary values of the Rayleigh quotient are precisely the eigenvalues of A.

2. Let K_n be the nth Krylov subspace generated in the Lanczos iteration, and let

$$Q_n = [q_1, \ldots, q_n]$$

be an orthonormal basis of K_n. The Lanczos process produces the tridiagonal matrix

$$T_n = Q_n^T A Q_n.$$

The Ritz values at step n are the eigenvalues of T_n.

Now restrict the Rayleigh quotient to vectors in K_n. Any nonzero $x \in K_n$ can be written as

$$x = Q_n y, \qquad y \in \mathbb{R}^n,\ y \neq 0.$$

Then

$$r(x) = r(Q_n y) = \frac{(Q_n y)^T A(Q_n y)}{(Q_n y)^T (Q_n y)}.$$

Since the columns of Q_n are orthonormal, $Q_n^T Q_n = I$, so

$$(Q_n y)^T (Q_n y) = y^T y,$$

and

$$(Q_n y)^T A(Q_n y) = y^T Q_n^T A Q_n y = y^T T_n y.$$

Therefore,

$$r(Q_n y) = \frac{y^T T_n y}{y^T y}.$$

Thus the restriction of the Rayleigh quotient to K_n is exactly the Rayleigh quotient of the symmetric matrix T_n. By part (a), the stationary values of

$$\frac{y^T T_n y}{y^T y}$$

are precisely the eigenvalues of T_n.

Since these eigenvalues are the Ritz values at step n, it follows that the Ritz values are exactly the stationary values of $r(x)$ when x is restricted to K_n.

$\square$

PROBLEM X The conjugate gradient method is applied to a symmetric positive definite matrix A. It is observed that

$$\|e_0\|_A = 1, \qquad \|e_{10}\|_A = 2 \times 2^{-10}.$$

Based solely on this data, answer the following questions:

(a) What bound can you give on the condition number $\kappa(A)$?

(b) What bound can you give on $\|e_{20}\|_A$?

SOLUTION. For the conjugate gradient method applied to a symmetric positive definite matrix A, the standard error bound in the A-norm is

$$\|e_k\|_A \leq 2 \left(\frac{\sqrt{\kappa(A)} - 1}{\sqrt{\kappa(A)} + 1} \right)^k \|e_0\|_A,$$

where

$$\kappa(A) = \frac{\lambda_{\max}(A)}{\lambda_{\min}(A)}.$$

We are given

$$\|e_0\|_A = 1, \qquad \|e_{10}\|_A = 2 \cdot 2^{-10}.$$

1. Using the CG error bound with $k = 10$, we obtain

$$\|e_{10}\|_A \leq 2 \left(\frac{\sqrt{\kappa(A)} - 1}{\sqrt{\kappa(A)} + 1} \right)^{10} \|e_0\|_A.$$

Substituting the given data,

$$2 \cdot 2^{-10} \leq 2 \left(\frac{\sqrt{\kappa(A)} - 1}{\sqrt{\kappa(A)} + 1} \right)^{10}.$$

Dividing both sides by 2 gives

$$2^{-10} \leq \left(\frac{\sqrt{\kappa(A)} - 1}{\sqrt{\kappa(A)} + 1} \right)^{10}.$$

Taking the tenth root,

$$\frac{1}{2} \leq \frac{\sqrt{\kappa(A)} - 1}{\sqrt{\kappa(A)} + 1}.$$

Let

$$t = \sqrt{\kappa(A)}.$$

Then

$$\frac{1}{2} \leq \frac{t-1}{t+1}.$$

Solving for t,

$$\frac{1}{2}(t+1) \leq t - 1,$$

$$\frac{1}{2}t + \frac{1}{2} \le t - 1,$$
$$\frac{3}{2} \le \frac{1}{2}t,$$
$$t \ge 3.$$

Hence

$$\sqrt{\kappa(A)} \ge 3, \qquad \kappa(A) \ge 9.$$

Therefore, based on the given data, we can conclude that

$$\boxed{\kappa(A) \ge 9.}$$

2. From part (a), the convergence factor satisfies

$$\frac{\sqrt{\kappa(A)} - 1}{\sqrt{\kappa(A)} + 1} \ge \frac{1}{2}.$$

Using the same CG bound for $k = 20$,

$$\|e_{20}\|_A \le 2 \left(\frac{\sqrt{\kappa(A)} - 1}{\sqrt{\kappa(A)} + 1} \right)^{20} \|e_0\|_A.$$

Using the value determined from the data,

$$\|e_{20}\|_A \le 2 \left(\frac{1}{2} \right)^{20} = 2^{1-20} = 2^{-19}.$$

Thus,

$$\boxed{\|e_{20}\|_A \le 2^{-19}.}$$

□

PROBLEM XI Suppose A is a dense symmetric positive definite 1000×1000 matrix with condition number $\kappa(A) = 100$. Estimate roughly how many floating-point operations (flops) are required to solve the linear system $Ax = b$ to a ten-digit accuracy by:

(a) the method of steepest descent;

(b) the conjugate gradient method.

SOLUTION. We assume that "ten-digit accuracy" means reducing the relative error by a factor of about

$$10^{-10}.$$

Since A is dense and 1000×1000, the dominant cost in each iteration is one matrix-vector multiplication, which requires approximately

$$2n^2 = 2(1000)^2 = 2 \times 10^6$$

flops. The additional inner products and vector updates cost only $O(n)$ flops and are negligible in comparison.

(a) **Steepest descent**

For steepest descent on an SPD matrix,

$$\|e_k\|_A \leq \left(\frac{\kappa(A)-1}{\kappa(A)+1}\right)^k \|e_0\|_A.$$

Here $\kappa(A) = 100$, so

$$\frac{\kappa-1}{\kappa+1} = \frac{99}{101} \approx 0.9802.$$

We want

$$\left(\frac{99}{101}\right)^k \approx 10^{-10}.$$

Taking logarithms,

$$k \approx \frac{\log(10^{-10})}{\log(99/101)} \approx 1151.$$

So steepest descent requires roughly 1.15×10^3 iterations.

Hence the total flop count is approximately

$$1151 \cdot 2 \times 10^6 \approx 2.3 \times 10^9 \text{ flops.}$$

$$\boxed{\text{Steepest descent: about } 2.3 \times 10^9 \text{ flops.}}$$

(b) **Conjugate gradient method**

For conjugate gradient,

$$\|e_k\|_A \leq 2\left(\frac{\sqrt{\kappa(A)}-1}{\sqrt{\kappa(A)}+1}\right)^k \|e_0\|_A.$$

Since $\kappa(A) = 100$, we have $\sqrt{\kappa} = 10$, so

$$\frac{\sqrt{\kappa}-1}{\sqrt{\kappa}+1} = \frac{9}{11} \approx 0.8182.$$

We want

$$2\left(\frac{9}{11}\right)^k \approx 10^{-10},$$

so

$$\left(\frac{9}{11}\right)^k \approx 5 \times 10^{-11}.$$

Taking logarithms,

$$k \approx \frac{\log(5 \times 10^{-11})}{\log(9/11)} \approx 118.$$

Thus CG requires roughly 1.2×10^2 iterations.

Hence the total flop count is approximately

$$118 \cdot 2 \times 10^6 \approx 2.4 \times 10^8 \text{ flops.}$$

$$\boxed{\text{Conjugate gradient: about } 2.4 \times 10^8 \text{ flops.}}$$

Therefore, a rough estimate is therefore:

$$\boxed{\begin{array}{r}\text{(a) Steepest descent} \approx 2.3 \times 10^9 \text{ flops,} \\ \text{(b) Conjugate gradient} \approx 2.4 \times 10^8 \text{ flops.}\end{array}}$$

□

PROBLEM XII Let A be the 100×100 tridiagonal symmetric matrix with $1, 2, \ldots, 100$ on the diagonal and 1 on the sub- and superdiagonals. Let

$$b = (1, 1, \ldots, 1)^T.$$

(a) Write a program that performs 100 steps of both the conjugate gradient (CG) method and the method of steepest descent to approximately solve $Ax = b$ with initial guess $x_0 = 0$.

(b) Produce a plot containing four curves:

- the computed residual norms $\|r_n\|_2$ for CG,
- the actual residual norms $\|b - Ax_n\|_2$ for CG,
- the residual norms $\|r_n\|_2$ for steepest descent,
- the estimate of the convergence rate

$$2\left(\frac{\sqrt{\kappa}-1}{\sqrt{\kappa}+1}\right)^n.$$

(c) Comment on your results.

SOLUTION. The convergence behavior of the conjugate gradient (CG) method and the method of steepest descent is illustrated in Figure 1. The CG method exhibits a significantly faster convergence rate compared to steepest descent. After an initial phase, the residual norm of CG decreases rapidly at an approximately exponential rate, which is consistent with theoretical predictions.

The computed residual $\|r_n\|_2$ and the true residual $\|b - Ax_n\|_2$ for CG are nearly identical throughout the iterations. This indicates that the accumulation of floating-point errors is negligible in this experiment and that the recursive computation of the residual remains reliable.

In contrast, the method of steepest descent converges much more slowly. Its residual norm decreases gradually and does not exhibit the same rapid decay as CG. This behavior reflects the well-known dependence of steepest descent on the condition number of the matrix.

The theoretical bound

$$2\left(\frac{\sqrt{\kappa}-1}{\sqrt{\kappa}+1}\right)^n$$

provides an upper bound on the convergence rate of CG. As shown in the figure, this bound is quite conservative, as the actual convergence of CG is significantly faster. This is because the bound depends only on the condition number and represents a worst-case scenario, while the actual eigenvalue distribution of the matrix is more favorable. □

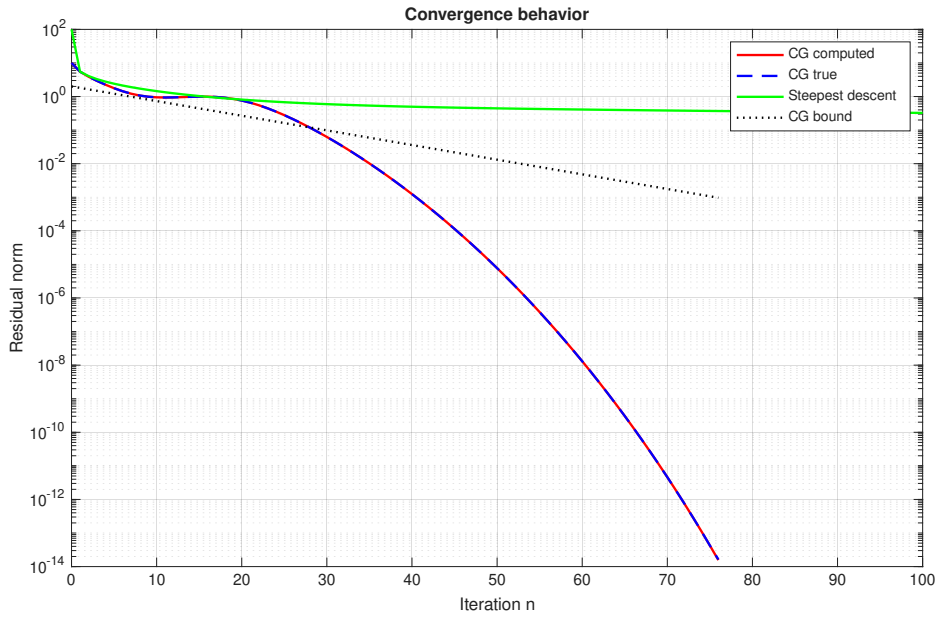

图 1: Convergence behavior of the conjugate gradient method and the method of steepest descent for solving $Ax = b$. The figure shows the computed residual $\|r_n\|_2$ and the true residual $\|b - Ax_n\|_2$ for CG, the residual norm for steepest descent, and the theoretical convergence bound.